

AI Security Stack Mapper

Map, assess & secure every layer of an LLM application stack · Working MVP · 2026-06-22

50

GRADE F

1. The Problem

Organizations are shipping LLM-powered applications faster than they can secure them. Unlike traditional apps, an LLM application has a **wide, layered attack surface**: a single user request flows through input handling, retrieval (RAG), system prompts and secrets, the model, tool/agent execution, output rendering, delivery, and monitoring. Each layer introduces distinct risks — prompt injection, jailbreaks, sensitive-data leakage, unsafe tool execution, retrieval poisoning, insecure output handling — and most teams have **no single place** to see which layers are protected, where the gaps are, and what to fix first.

2. The Solution

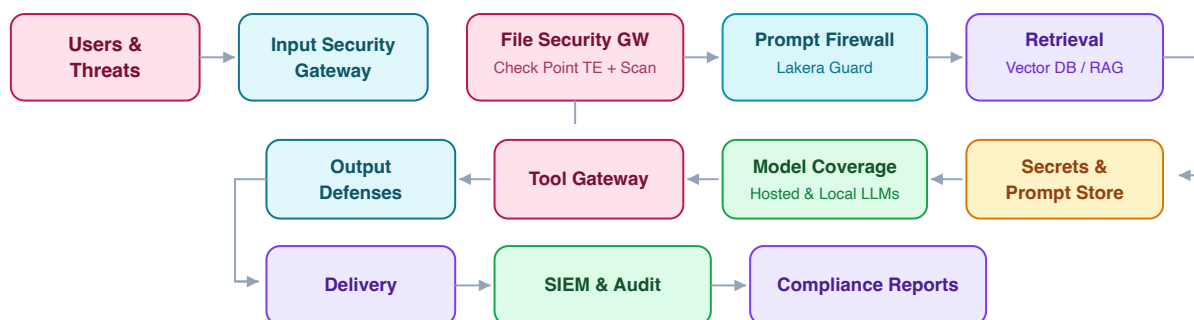
The **AI Security Stack Mapper** is a SOC-style dashboard that lets a team map, assess, and secure every layer of an LLM application stack. It models the application as a **defense-in-depth pipeline**, visualizes how prompts and files flow through security controls, shows where attacks can happen, and tracks which controls mitigate each risk — producing a single **AI Security Score** and an exportable assessment report.

It ships with realistic demo data and integrates **Check Point** Threat Emulation / TE File Scan (file security) and **Lakera** Guard (prompt firewall), plus full LLM coverage management for hosted and local models.

One-line value: turn “we have an LLM app somewhere in production” into a measured, layer-by-layer security posture with a clear remediation plan.

3. How It Works — the Defense-in-Depth Flow

Every request and ingested document is treated as untrusted and passes through a sequence of security zones. The app maps controls onto each zone:



Flow: user input & files enter (top) → model & tools (middle) → delivery, audit & compliance (bottom).

Check Point scans files before the RAG corpus; **Lakera Guard** screens prompts & outputs.

Malicious / suspicious files are **blocked from retrieval**; risky prompts are blocked, redacted or escalated.

- **Files** (uploads, KB documents, attachments) are detonated/scored by **Check Point** before they enter the vector DB; malicious or suspicious verdicts are **blocked from the RAG corpus**.
- **Prompts** are screened by **Lakera** Guard for injection, jailbreak, unsafe content and leakage — returning a risk score and a decision (**allow / block / redact / escalate**). Outputs can be screened the same way.
- **Models** (OpenAI, Azure, Anthropic, Gemini, Llama, Mistral, Cohere, local runtimes, custom) are registered with their guardrail and isolation posture.
- **Every event** is designed to flow to SIEM/audit for detection and compliance reporting.

4. What a User Does (Workflow)

1. **Dashboard** — overall AI Security Score, protected layers, open risks, critical gaps, vendor-protection metrics, recent incidents.
2. **LLM Stack Map** — interactive defense-in-depth diagram; nodes color-coded by status; click to inspect/edit.
3. **Layer Assessment** — mark each control *Not implemented* → *Planned* → *Implemented* → *Verified*.
4. **Risk Register** — track LLM threats, severity, and mitigating controls (mapped to OWASP Top 10 for LLM Apps).
5. **Control Library / Architecture Builder** — browse, filter, add, or drag controls onto the canvas.
6. **Integrations** — File Security Gateway, Lakera Guard, and Model Coverage do the live (mock) work.
7. **Report Generator** — export an executive report (Markdown or PDF) with findings, gaps and a remediation plan.

5. The AI Security Score

A single 0–100 score (graded A–F) combines:

Weighted coverage

Credit scales with control maturity: Planned 25% → Implemented 75% → Verified 100%.

Bonuses (+)

- Check Point file scanning enabled
- Lakera Guard input / output protection
- Fully-guardrailed production models
- Network-isolated local LLMs
- Per-model tool restriction

Penalties (–)

- Files entering RAG unscanned
- Missing logging, output moderation, tool restriction or secrets management
- Models missing input / output guardrails
- Local LLMs with internet access or unrestricted tools
- Unresolved critical risks

The score updates **live** as controls, integrations and model posture change, so remediation progress is immediately visible.

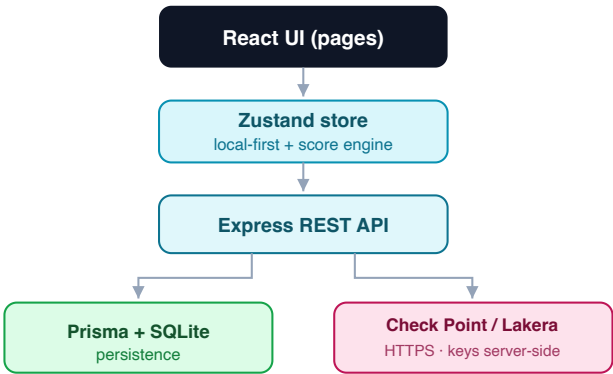
6. Data Model

ENTITY	REPRESENTS
Project	An LLM application under assessment
StackLayer	A defense-in-depth zone (input, file, model, ...)
SecurityControl	A mitigation, mapped to a layer + OWASP category
Risk	An LLM-specific threat with severity & mitigating controls
Assessment	A control's implementation status over time
Evidence	Proof a control is in place
Incident	A detected security event
ComplianceMapping	OWASP LLM Top-10 coverage status
Integration	A vendor connector (Check Point, Lakera) + health
ModelCoverage	A registered hosted/local LLM and its posture
FileScan	A file scan record + verdict + evidence
PromptInspection	A Lakera Guard prompt/output verdict

7. Architecture & Technology

Frontend — React 18 + TypeScript (Vite), Tailwind CSS + DaisyUI
dark SOC theme, React Flow for the interactive map/builder, Zustand
state with local-first persistence.

Backend — Node.js + Express REST API, Prisma ORM over SQLite.
Vendor integration logic runs **server-side**, so API keys never reach
the browser.



Security & data handling

- **No hardcoded secrets.** Keys are read only from environment variables; until configured, integrations run in **mock mode** with no network calls.
- **HTTPS only;** TLS verification is never disabled.
- Files are hashed locally (SHA-256); no bytes leave the client in mock mode.
- Vendor calls are proxied through the backend, keeping credentials server-side.

8. Status & Next Steps

Delivered (MVP): all dashboards and screens, interactive stack map, risk register, control library, architecture builder, report export, the Check Point + Lakera + model-coverage integration layer, the live security score, seeded demo data, and a working Express + Prisma + SQLite backend the frontend hydrates from (with offline fallback).

Recommended next steps

1. **Authentication & RBAC** so SOC / AppSec / owners have scoped access.
2. **Live vendor credentials** via the backend env config (already wired for mock→live switchover).
3. **Live SIEM forwarding** and automated SOC ticket creation.
4. **Multi-user deployment** (managed Postgres) and audit history.
5. **CI gate** that fails a deploy if a project's score drops below threshold.

